Graph-Based Mule Ring Detection System

One-liner: A fraud intelligence system that models UPI-style transactions as a dynamic account network and detects money-mule laundering rings — instead of scoring transactions one at a time.

Why this project (the pitch)

Most student fraud projects ask: *"Is this transaction fraudulent?"* This project asks: *"Is this account part of a laundering network?"*

That's the difference between a classroom ML exercise and something that resembles a real fraud intelligence system — and it's inspired by publicly reported mule-account detection patterns used across the fintech fraud space, not a claim of building anything used by RBI/NPCI.

Resume / interview line:

Built a graph-based fraud intelligence system modeling UPI-style transactions as a dynamic account network, using rule-based and ML risk scoring to detect multi-hop mule laundering rings — validated against synthetic fraud rings injected into 10,000+ simulated accounts.

Core concept

- **Account = node, transaction = directed edge**
- Fraud signal isn't in one transaction — it's in the *shape and timing* of a chain: Dormant account → sudden large credit → forwarded within minutes → repeated 4-6 hops → cash withdrawal
- Detecting this requires graph structure + time, not a flat feature table.

Architecture (v1 — deliberately lean)

Transaction Generator (synthetic, with injected mule rings)

↓

Neo4j Graph Store

↓

Rule-Based Detector → Feature Extraction → XGBoost Risk Model

↓

FastAPI Scoring Endpoint

↓

React Dashboard (shows flagged ring + money-flow path)

Explicitly out of scope for v1 (state this openly in the writeup — it signals judgment, not gaps):

- Redis — only add later if load testing shows Neo4j query latency is the actual bottleneck; add it as a measured decision, not a default
- Kafka — a simple queue is enough to prove the streaming concept
- GNN — start with interpretable graph features (degree, PageRank-style risk propagation, hop-distance); add a GNN only if time remains and you can explain it

---

3-Week Build Plan

Week	Deliverable
1	Synthetic account/transaction generator (normal patterns + injected mule rings) → loaded into Neo4j → working rule-based detector (dormancy + velocity + forwarding-ratio rules)
2	Feature extraction (transaction velocity, dormancy period, hop-distance, PageRank-style risk propagation) → XGBoost model layered on top of rules → FastAPI scoring endpoint
3	React dashboard visualizing a flagged ring and its money-flow path → load test graph queries (p99 latency) → write up results and honest tradeoffs

---

Detection logic (rule baseline — build this first)

IF account dormant > 180 days

AND receives > 10 transactions in 10 minutes

AND forwards > 80% of received amount within 15 minutes

THEN flag as High Risk

ML layer adds features on top: node_degree, transaction_velocity, average_hop_distance, account_age, amount_variance, dormancy_period.

The demo that matters

Interviewers won't remember the architecture diagram. They'll remember:

"I generated 10,000 synthetic accounts, injected one mule ring, and the system caught it — here's the dashboard showing the flagged path: Account A → B → C → D → Withdrawal, risk score 94%, reasons: dormant account, high velocity, multi-hop forwarding."

Build the dashboard around this single moment.

Talking points for interviews

- **Why graph over tabular:** single-transaction classifiers miss network-level laundering patterns that only appear across multiple hops and accounts.
- **Why rules before ML:** interpretable baseline first, ML adds lift on top — shows engineering judgment, not just model-chasing.
- **Why no GNN in v1:** avoided using a technique you can't fully explain under interview pressure; named as future work instead.
- **Why no Redis/Kafka in v1:** avoided adding infrastructure without a measured reason; framed as "add only if profiling justifies it."
- **Grounding in reality:** reference the real scale of the problem (hundreds of thousands of mule accounts flagged in India in 2026, regulatory action underway) to show you understood the actual domain, not just a Kaggle dataset.